

Paradygmaty Programowania

Laboratorium 4

Programowanie współbieżne

1 Wstęp teoretyczny

Przed przeczytaniem niniejszego wstępu zapoznaj się z materiałami do wykładu 8.

1.1 Cel ćwiczenia

Celem ćwiczenia jest zaznajomienie studentów z wybranymi zagadnieniami zrównoleglania obliczeń i przetwarzania współbieżnego.

1.2 UML i wzorce projektowe – przypomnienie

Przetwarzanie współbieżne to technika polegająca na wykonywaniu kilku operacji jednocześnie w celu przyspieszenia obliczeń. Istnieją różne sposoby zrównoleglania:

- Wątki (Threads): Dzielą proces na mniejsze jednostki, które mogą być wykonywane równocześnie. Wątki współdzielą tę samą przestrzeń adresową, co ułatwia komunikację, ale wymaga synchronizacji dostępu do wspólnych zasobów.
- Procesy (Processes): Każdy proces działa w swojej własnej przestrzeni adresowej. Procesy są bardziej izolowane od siebie, co zmniejsza ryzyko błędów synchronizacji, ale komunikacja między nimi jest trudniejsza.
- Aktorzy (Actors): Model programowania, w którym jednostki obliczeniowe (aktorzy) komunikują się za pomocą wymiany wiadomości. Każdy aktor ma swój własny stan i przetwarza wiadomości asynchronicznie.
- Kolejki zadań (Task Queues): Technika polegająca na rozdzieleniu zadań do wykonania między różne jednostki obliczeniowe, które przetwarzają zadania równocześnie.

2 Przykłady problemów zrównoleglalnych i jak były zrównoleglane

2.1 Sortowanie dużych zbiorów danych

Algorytm QuickSort: Może być zrównoleglony poprzez jednoczesne sortowanie podzbiorów danych.

Implementacja wątkowa: Każdy wątek sortuje inny podzbiór, a następnie wyniki są scalane.

2.2 Przetwarzanie obrazów

Filtry obrazu: Można zrównoleglić przez równoczesne przetwarzanie różnych fragmentów obrazu.

Implementacja wątkowa: Każdy wątek przetwarza inną część obrazu, a wyniki są scalane w końcowym obrazie.

2.3 Symulacje fizyczne

Symulacje cząsteczek: Można zrównoleglić przez równoczesne obliczenia sił działających na różne cząsteczki.

Model aktorów: Każda cząsteczka jest aktorem, który wymienia wiadomości z innymi aktorami, aby obliczyć siły i ruch.

3 Przebieg ćwiczenia

Termin oraz sposób dostarczania sprawozdań należy uzgodnić z prowadzącym(i) zajęcia. Sprawozdanie powinno zawierać: **imię i nazwisko, numer albumu, datę**.

Ćwiczenie przebiega według następującego harmonogramu:

- przedstawienie problemu programistycznego
- wspomagane przez prowadzących rozwiązywanie zadania

Zadanie polega na narysowaniu odpowiedniego diagramu na podstawie analizy przygotowanego przez prowadzących kodu i zaproponowaniu sposobu przyspieszenia obliczeń przy wykorzystaniu przetwarzania współbieżnego. Następnie na podstawie zaproponowanych mechanizmów należy przygotować diagram zmodyfikowanego algorytmu.

4 Przykładowe zadanie : Mnożenie dużych macierzy

Poniżej przedstawiono kod programu mnożącego duże macierze liczbowe. Należy założyć, że naiwny algorytm (i nieoptymalny z punktu widzenia zastosowania języka Matlab) może być w przyszłości wykorzystany do mnożenia macierzy w innych ciałach liczbowych (np. modulo N) lub innych nieliczbowych obiektów.

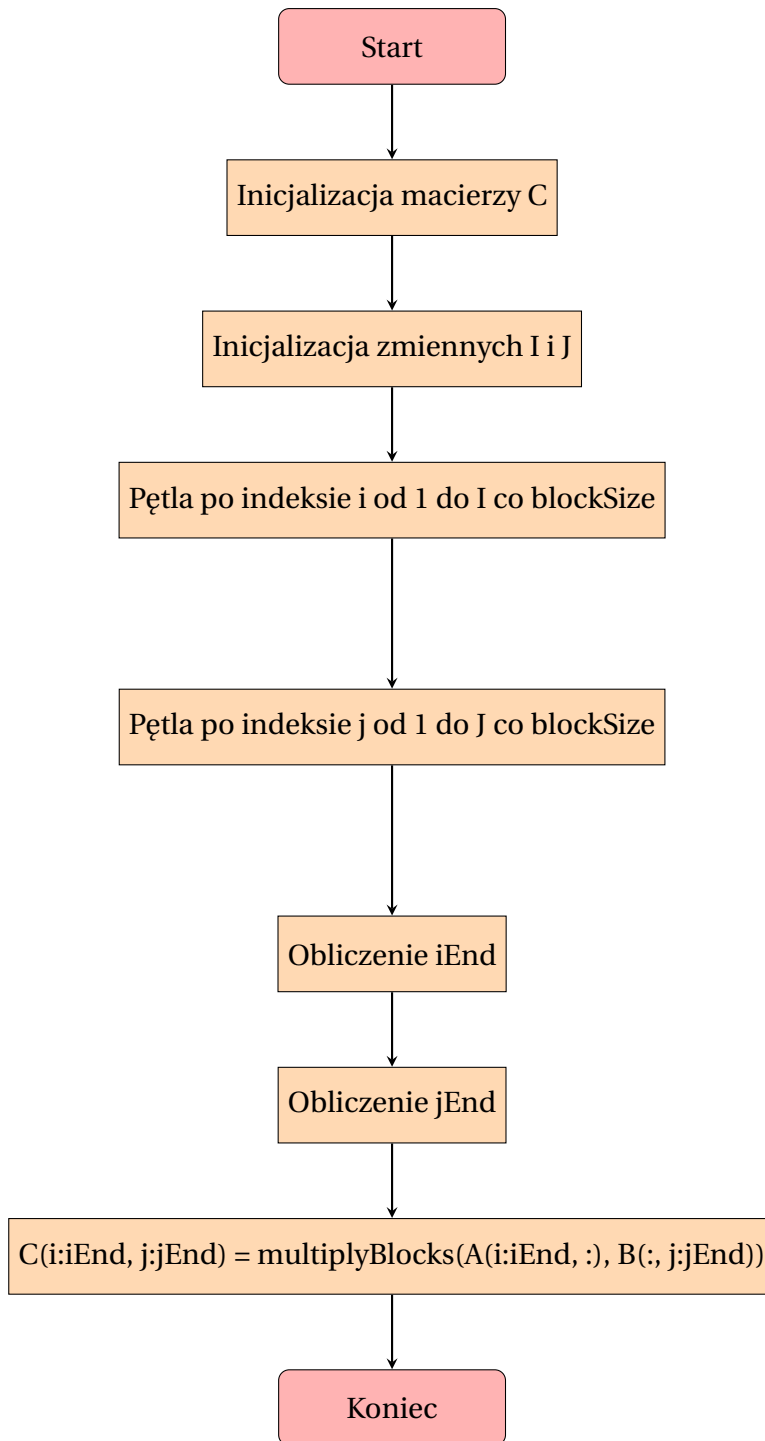
```
function C = multiplyWithoutThreads(A, B, blockSize)
    C = zeros(size(A,1), size(B,2), 'like', A);

    I = size(A,1);
    J = size(B,2);

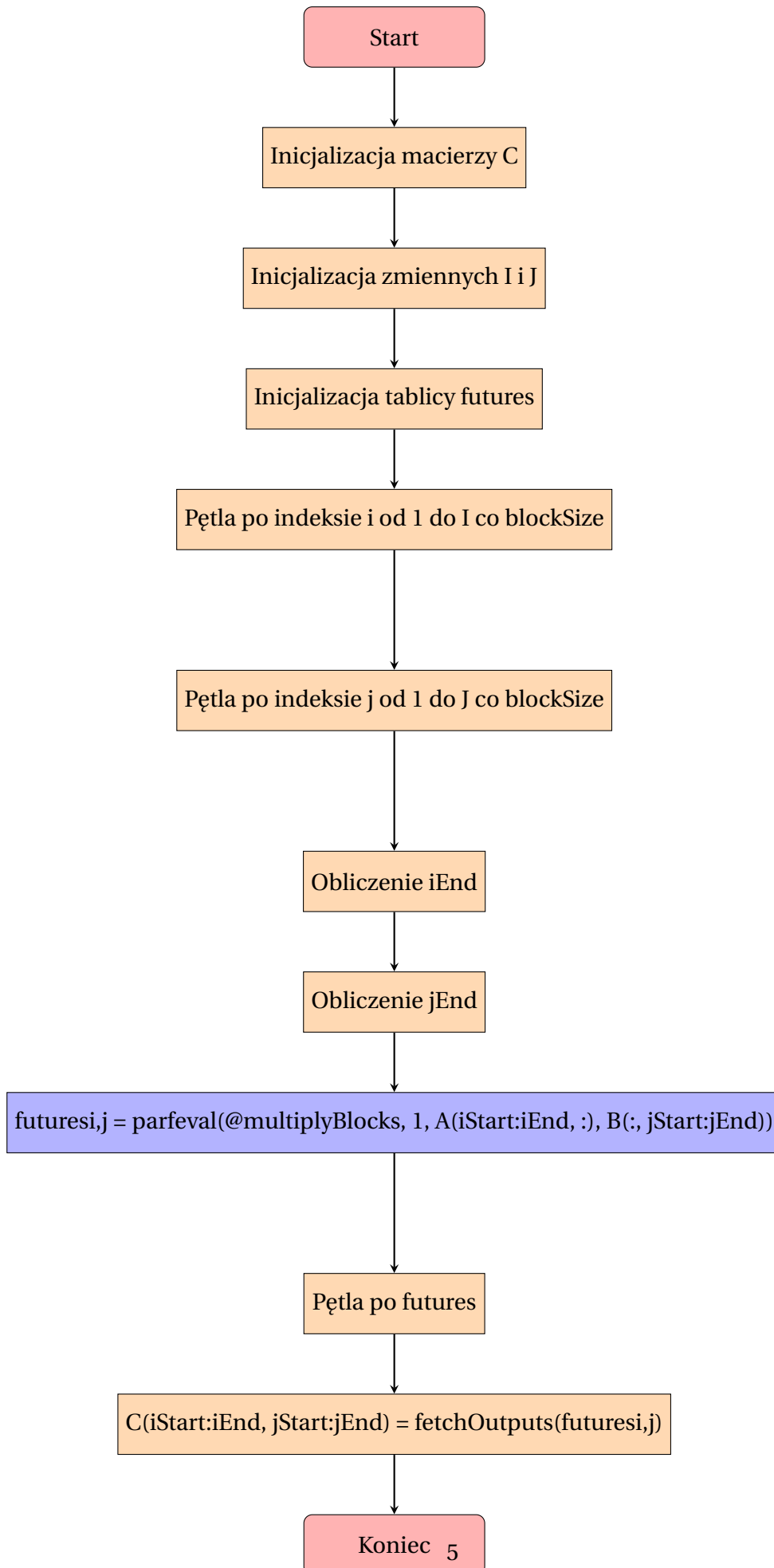
    for i = 1:blockSize:I
        for j = 1:blockSize:J
            iEnd = min(i+blockSize-1, I);
            jEnd = min(j+blockSize-1, J);

            C(i:iEnd, j:jEnd) = multiplyBlocks(A(i:iEnd, :), B(:, j:jEnd));
        end
    end
end
```

W powyższym kodzie macierze wejściowe podzielone zostały na podmacierze, a następnie wszystkie fragmenty zostały pomnożone przez siebie i zapisane we wspólnym wyniku. Takie rozwiązanie może być lepsze z punktu widzenia wykorzystania pamięci podręcznej i często wykorzystywane jest do implementacji mnożenia macierzy w przypadku silnie zrównolegowanego algorytmu, np. z wykorzystaniem kart graficznych.



Analizując algorytm można zauważyć, że wywołania funkcji mnożącej poszczególne bloki nie zależą od siebie i mogą być wywołane równolegle. Nie występuje zależność danych która prowadziłaby do ograniczenia możliwości zrównoleglania. Poniżej zaproponowano diagram dla zrównoleglonej wersji algorytmu i odpowiadający mu kod w języku Matlab. (Uwaga: Studenci w trakcie zajęć nie będą pisali własnych zrównoleglonych wersji algorytmów. Poniższy kod jest jedynie przykładem implementacji).



```

function C = multiplyWithThreads(A, B, blockSize)
    C = zeros(size(A,1), size(B,2), 'like', A);

    I = size(A,1);
    J = size(B,2);

    futures = cell(ceil(I/blockSize), ceil(J/blockSize));

    for i = 1:size(futures,1)
        for j = 1:size(futures,2)
            iStart = (i-1) * blockSize + 1;
            jStart = (j-1) * blockSize + 1;

            iEnd = min(iStart + blockSize - 1, I);
            jEnd = min(jStart + blockSize - 1, J);

            futures{i,j} = parfeval(@multiplyBlocks, 1, A(iStart:iEnd, :),
                                    B(:, jStart:jEnd));
        end
    end

    for i = 1:size(futures,1)
        for j = 1:size(futures,2)
            iStart = (i-1) * blockSize + 1;
            jStart = (j-1) * blockSize + 1;

            iEnd = min(iStart + blockSize - 1, I);
            jEnd = min(jStart + blockSize - 1, J);

            C(iStart:iEnd, jStart:jEnd) = fetchOutputs(futures{i,j});
        end
    end
end

```

```

function C = multiplyBlocks(A, B)
    C = A * B;
end

```

Przedstawione algorytmy ilustrują różnice między przetwarzaniem sekwencyjnym a równoległym. Algorytm z przetwarzaniem współbieżnym pozwala na znaczną optymalizację czasu obliczeń przez równoległe wykonywanie mnożenia bloków macierzy. Wykorzystanie funkcji `parfeval` w Matlabie umożliwia łatwe zrównoleglenie zadań i efektywne zarządzanie wynikami obliczeń równoległych.

Ostatnia aktualizacja: 29 kwietnia 2026